



微机原理及接口技术

第一次大作业报告

学 生 学 号	22375413
学 生 姓 名	夏 思 博
指 导 教 师	李 妮

2024 年 11 月

1 任务内容

表 1 第一次大作业任务内容

任务	任务名称	任务内容
1	冒泡排序	从键盘输入(回车符结束 0DH), 从小到大冒泡排序并显示
2	四位十进制数加法	从键盘输入 2 个四位十进制数(X1, X2), 相加并显示十进制结果, 并去掉起始的 0

2 实现原理

任务一：通过不断读取键盘输入，直到用户输入回车符，将每个字符存储到内存中的数组里。利用冒泡排序算法，通过重复遍历数组，比较并交换相邻元素，直到数组中的元素从小到大排序。遍历已排序的数组，将每个元素显示在屏幕上。

任务二：通过 DOS 中断读取用户输入的每个字符，并将它们转换为数值存储。将字符形式的十进制数转换为数值形式，以便进行计算。逐位相加两个数值，处理进位，并存储结果。检查并去掉结果中不必要的起始 0。将计算结果转换回字符形式，并显示在屏幕上。

3 汇编源程序

3.1 任务一源程序及分析

输入过程：读取键盘输入到 AL 寄存器。如果输入的是回车符（0DH），则结束输入。

结束输入：在输入结束后，将 SI 指向的位置设置为 0，作为字符串的结束标志。将字符数量（CX）存储到 BX 寄存器，用于后续的排序循环。

排序过程：使用 BX 作为循环次数，控制排序的外部循环。使用 CX 作为循环次数，控制排序的内部循环。如果当前字符（AL）大于下一个字符（DL），则交换这两个字符。从 X1 的起始位置开始，将每个字符输出到屏幕，直到遇到字符串结束符（0）。

其中，输入\$即为其对应的 ASCII 码，不受任何影响。

代码注释如下：

```

0001 STACK SEGMENT STACK
0002     DW 100 DUP(?)
0003 STACK ENDS
0004
0005 DATA SEGMENT
0006     X1 DB 100 DUP(?)
0007 DATA ENDS
0008
0009 CODE SEGMENT
0010 ASSUME CS:CODE, DS:DATA, SS:STACK
0011 START:
0012     MOV AX, DATA
0013     MOV DS, AX
0014     MOV SI, OFFSET X1;将SI指向X1
0015
0016 INPUT:
0017     MOV AH, 01H ; 读取键盘输入，存入数据段的X1中
0018     INT 21H
0019     CMP AL, 0DH ; 检查回车符结束输入
0020     JE END_INPUT; 移除对'$'的检查，允许输入'$'
0021     MOV [SI], AL      ; 存储输入字符
0022     INC CX
0023     INC SI            ; 增加字符计数
0024     JMP INPUT
0025
0026 END_INPUT:
0027     MOV [SI], 0      ; 输入结束后添加结束符
0028     MOV BX, CX       ; 设置 BX 为字符数里
0029     DEC BX           ; 设置外循环次数，BX = 字符数 - 1
0030     MOV SI, OFFSET X1
0031
0032 OUTLOOP:
0033     MOV DI, OFFSET X1 ; 内层循环从头开始
0034     MOV CX, BX        ; 内层循环次数，逐渐减少
0035

```

```

0036 INLOOP:
0037     MOV AL, [DI]      ; AL = 当前字符
0038     MOV DL, [DI+1]    ; DL = 下一个字符
0039     CMP AL, DL
0040     JBE NO_SWAP       ; 如果当前字符小于等于下一个字符，不交换
0041     MOV [DI], DL      ; 否则交换
0042     MOV [DI+1], AL
0043
0044 NO_SWAP:
0045     INC DI            ; DI 指向下一个字符
0046     LOOP INLOOP      ; 重复内循环直到排序完成
0047     DEC BX           ; 减少外循环次数
0048     JNZ OUTLOOP      ; 重复外循环直到所有数字排序完成
0049
0050     MOV AH, 2         ; 输出换行
0051     MOV DL, 0DH
0052     INT 21H
0053     MOV DL, 0AH
0054     INT 21H
0055
0056     MOV SI, OFFSET X1 ; 从字符串起始位置输出排序后的字符
0057 OUTPUT:
0058     MOV AL, [SI]
0059     CMP AL, 0         ; 检查是否到达字符串结尾
0060     JE EXIT
0061     MOV DL, AL
0062     MOV AH, 2
0063     INT 21H
0064     INC SI
0065     JMP OUTPUT        ; 持续输出字符直到遇到结束符号
0066
0067 EXIT:
0068     MOV AH, 4CH
0069     INT 21H
0070 CODE ENDS
0071 END START

```

3.2 任务二源程序及分析

简要说就是把第一个四位数存进 X1，第二个四位数存进 X2，然后进行相加，巧妙运用 CF 进位算出加法结果存进 X3，重点在于把万位用 CF 提取出来，然后从万位开始输出 X3，依次与 0 比较，并用 AL 寄存器标志是否找到第一个非零位进行输出，注释如下：

```

0001 DATA SEGMENT
0002     X1 DB 4 DUP(?)
0003     X2 DB 4 DUP(?)
0004     X3 DB 5 DUP(?)
0005 DATA ENDS
0006
0007 STACK SEGMENT STACK
0008     DB 100 DUP(?)
0009 STACK ENDS
0010
0011 CODE SEGMENT
0012     ASSUME CS:CODE, DS:DATA, SS:STACK
0013 MAIN PROC
0014     MOV AX, DATA
0015     MOV DS, AX
0016
0017     MOV BX, OFFSET X1;将X1的四个字符依次存好
0018     MOV CX, 4
0019 NEXT:
0020     CALL KEYIN;输入字符ASCII码存在AL中并显示
0021     AND AL, 0FH;与00001111相与，将AL中的ASCII码变成非压缩型BCD码
0022     MOV [BX], AL
0023     INC BX;BX自加，循环结束后正好在X1第四个位置
0024     LOOP NEXT
0025
0026     MOV AH, 02H           ; 输出换行
0027     MOV DL, 0DH
0028     INT 21H
0029     MOV DL, 0AH
0030     INT 21H
0031
0032
0033     MOV BX, OFFSET X2;将X2的四个字符依次存好
0034     MOV CX, 4
0035 NEXT1:
0036     CALL KEYIN;输入字符ASCII码存在AL中并显示

```

```

0037      AND AL, 0FH;与00001111相与，将AL中的ASCII码变成非压缩型BCD码
0038      MOV [BX], AL;将AL放进以BX为地址的数据段位置，即X2中的某一位置
0039      INC BX;BX自加，循环结束后正好在X2第四个位置
0040      LOOP NEXT1;至此已将两个四位数放进数据段
0041
0042      MOV AH, 02H           : 输出换行
0043      MOV DL, 0DH
0044      INT 21H
0045      MOV DL, 0AH
0046      INT 21H
0047
0048
0049      MOV SI, OFFSET X1+3;将X1个位偏移地址放进SI
0050      MOV DI, OFFSET X2+3
0051      MOV BX, OFFSET X3+4
0052      MOV CX, 4;开始循环
0053      OR CX, CX;CX与自己或，CX不变，标志位CF清零
0054  NEXT2:
0055      MOV AL, [SI];X1个位放入AL
0056      MOV AH, 0
0057      ADC AL, [DI];X1个位和X2个位相加（其他位再加上前一位的进位CF），结果放入AL
0058      AAA;AX为结果的非压缩型BCD码，并记录进位CF
0059      MOV [BX], AL;结果个位放入X3个位
0060      DEC SI;SI自减，转为指向X2十位
0061      DEC DI
0062      DEC BX
0063      LOOP NEXT2
0064
0065      MOV AL, 0
0066      ADC AL, 0;将千位相加的进位CF取出放入AL
0067      MOV [BX], AL;将进位值放入X3万位，至此相加结果已放入X3数据段
0068
0069      MOV BX, OFFSET X3
0070      MOV CX, 5

```

```

0071     XOR AL, AL
0072 NEXT3:
0073     MOV DL, [BX]
0074     ADD DL, 30H
0075     CMP AL, 1
0076     JE SHOW
0077     CMP DL, 30H
0078     JE NOTSHOW
0079 SHOW:
0080     MOV AH, 2
0081     INT 21H
0082     MOV AL, 1
0083 NOTSHOW:
0084     INC BX
0085     LOOP NEXT3
0086
0087     MOV AH, 4CH
0088     INT 21H:退出
0089 MAIN ENDP
0090
0091 KEYIN PROC:判断输入程序并回显的子程序, 8号功能键入, 判断输入的内容ASCII是否'0'-'9', 是就回显
0092 AGAIN:
0093     MOV AH, 8
0094     INT 21H:将输入数字ASCII码放进AL
0095     CMP AL, 30H
0096     JB AGAIN:小于返回, 等待重新键入
0097     CMP AL, 39H
0098     JA AGAIN:大于返回, 等待重新键入
0099     MOV DL, AL
0100     MOV AH, 2
0101     INT 21H:二号功能显示DL中输入的数字
0102     RET
0103 KEYIN ENDP
0104
0105 CODE ENDS
0106     END MAIN

```

4 程序测试及结果

4.1 任务一程序测试

上一行 8273546271822 是输入, 下一行 1222234567788 是输出, 成功完成升序排列。

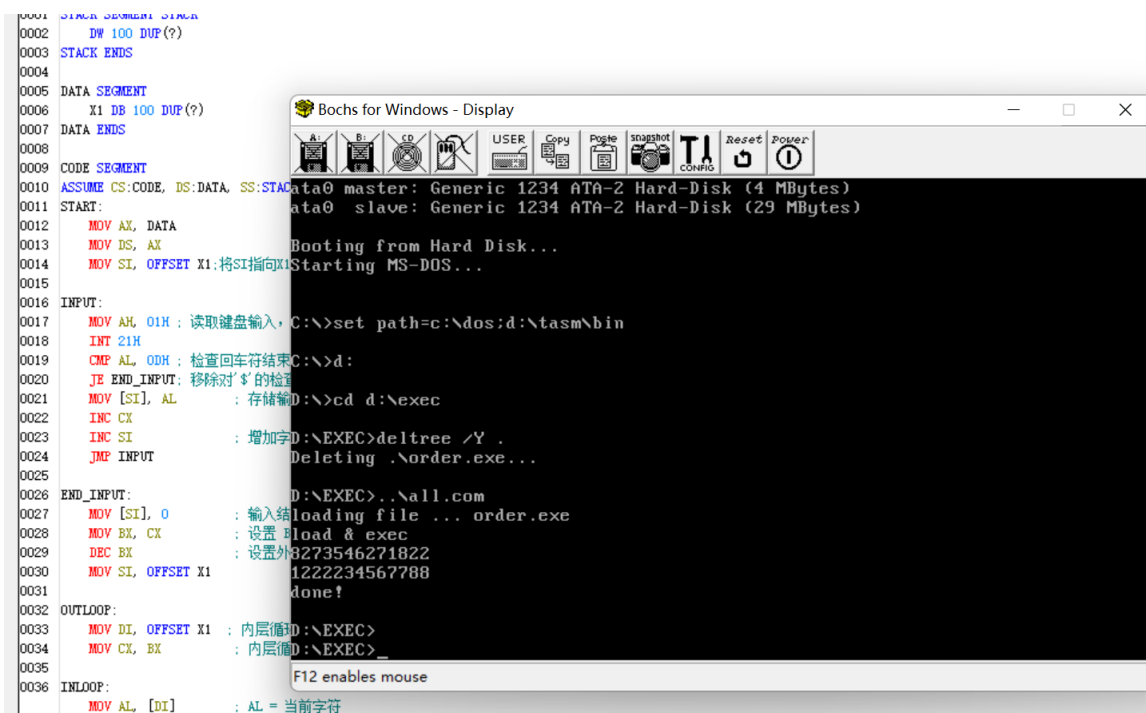


图 1 任务一冒泡排序案例结果

4.2 任务二程序测试

案例 1：中途含零且不进万位的案例 $1001+1001=2002$

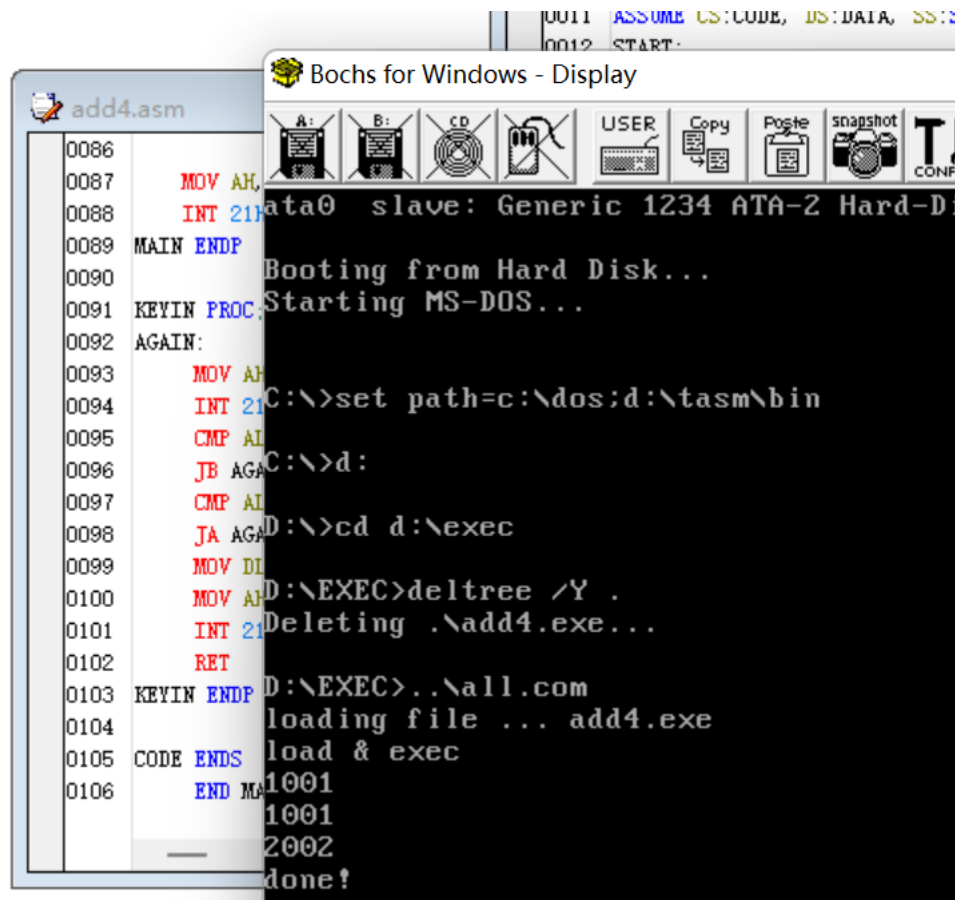


图 2 任务二无进位中间位含零案例

案例 2：含有进万位的案例 $5212+6011=11223$

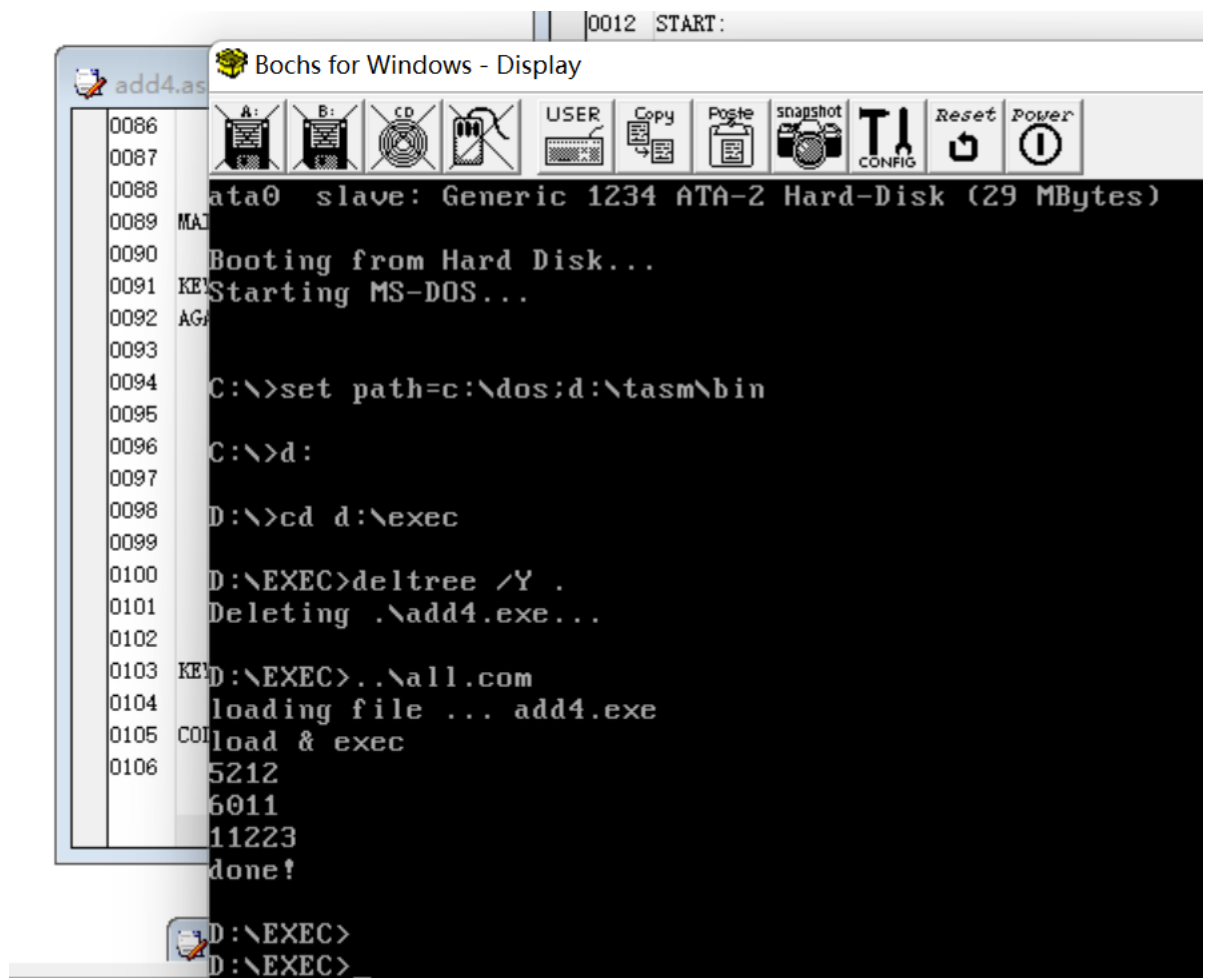


图 3 任务二进位案例

5 总结与思考

5.1 问题和困难

- 1、需修改编译器 TASM32，才能顺利地在自己的机器上完成编译。
- 2、遇到 INT 21H 不能单步执行，需打断点连续执行否则出不来无法键入。
- 3、文件路径不能含有空格
- 4、一开始使用电脑右侧数字键盘键入但 ASCII 码全为零无法键入，发现只能用键盘上方的数字 0-9 键入。
- 5、任务二中的结果要去掉起始的零，容易把结果中间的零也去掉，需要用 AL 寄存器记录第一个非零数，在第一个非零数的后面不再去除零位。

5.2 收获和体会

在本实验中我更加清晰地理解了什么是代码段、堆栈段和数据段。能够通过简单指令和 8086 提供的输入输出功能完成简单的加减法和排序。我觉得汇编语言代码还是需要自己多多练习，本质和 C 语言一样。一是要注意代码必须能运行不能出纰漏否则编译不了也没有办法单步执行 debug，二是我们可以在一串代码的前面加上伪代码如 output, input 等说明这串代码的功能，虽然循环和跳转不会用到，但整体提高了代码的可读性。

5.3 任务拓展

在任务二中，如果不去除前面的零可以如下写：

```
MOV BX, OFFSET X3
```

```
MOV CX, 5
```

```
NEXT3:
```

```
MOV DL, [BX]
```

```
ADD DL, 30H
```

```
MOV AH, 2
```

```
INT 21H
```

```
INC BX
```

```
LOOP NEXT3
```

报告中采用 AL 寄存器去标志代码是否已经读取到 X3 数据段中的非零位。

我曾尝试用 ZF 位作为非零位的检测，CMP DL, 30H 时，位数为零 ZF=1，因此可以 ZF=0 时跳过位数和 0 的比较，直接执行回显，但是有一点要注意在之前要让 ZF 归零，比如 CMP AX, AX